Llama Mobile

Luka Ribar
Graphcore

Jeevan Bhoot
Arm

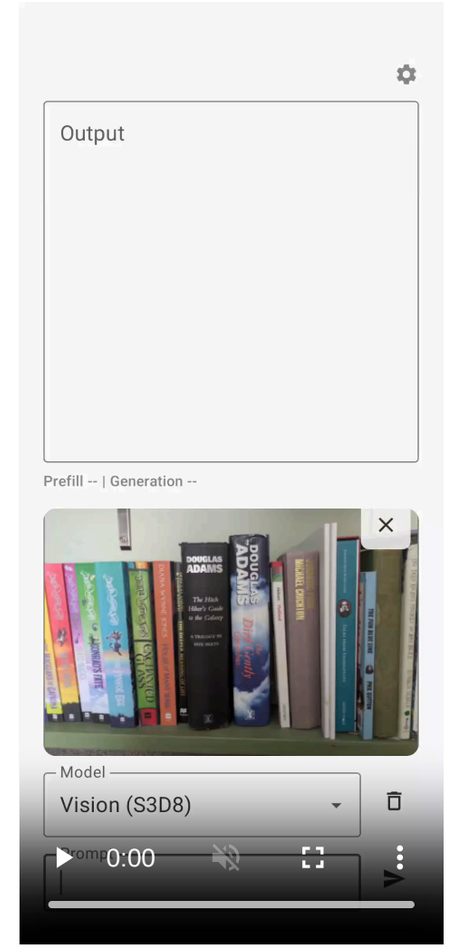
Douglas Orr
douglaso@graphcore.ai
Graphcore

[the paper, blog]

Goal: Compress a 11B parameter VLM to ~3.7 GB, for a phone

Towards this, we expected to work on weight quantization, requiring **high compression** (< 3 bit/parameter) & **fast decoding**.

- *Format-kernel codesign:* Arm-friendly S3D8 weight format
- *Fine-tuning recipe:* Multimodal QAT without pretraining data
- *Demo implementation:* Ground-up Linux/Android Arm CPU inference



Contents

- Problem setup
- Format design
- Quantisation-aware training
- Results
- Limitations & further work

Problem setup : Vision-Language Models

VLMs are LLMs with vision encoders to enable processing both images and text as inputs.

They are a natural fit to mobile deployment!

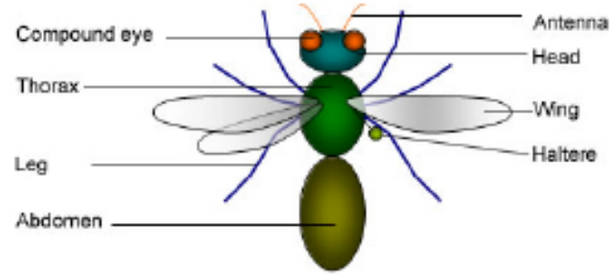
But, can be large, e.g. Llama 3.2 11B Vision Instruct is 22 GB in bfloat16.

Model Execution:

```
vision_cache = vision_encode(image, vision_params)
text_cache = text_prefill(vision_cache, query, text_params)

# repeatedly...
next_token, text_cache = \
  predict(vision_cache, text_cache, text_params)
```

Example (AI2D):



Query: Which of these is used for locomotion or support? Options: 1) Antenna, 2) Abdomen, 3) Leg, 4) Head.

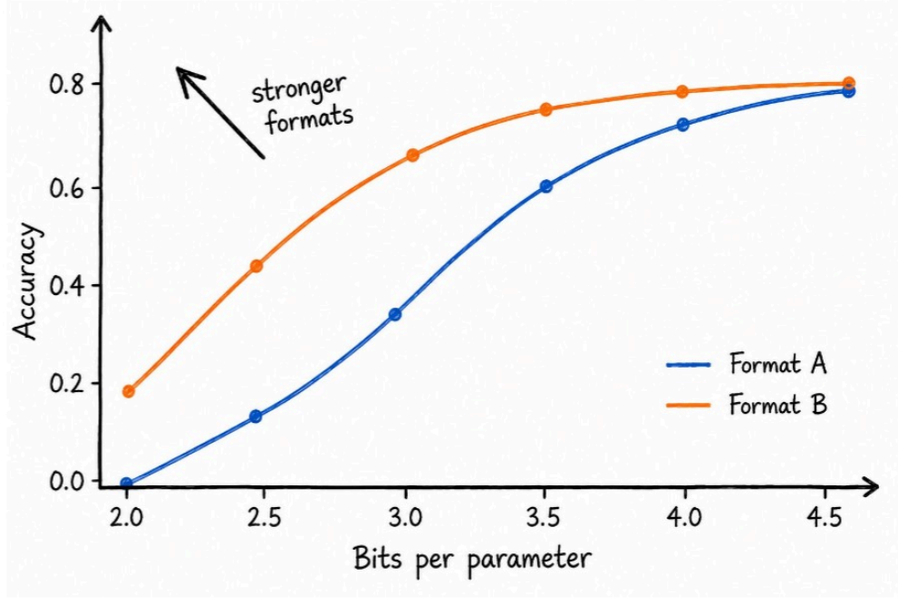
Expected answer: 3) Leg

Problem setup : Quantisation

To reduce model size, we focus on weight quantisation (other approaches include sparsity & distillation).

Three competing objectives:

1. **Accuracy** - minimise reduction in task performance
2. **Size** - compress the weights as much as possible
3. **Speed** - dequantize quickly; ideally next-token-prediction remains memory-bound

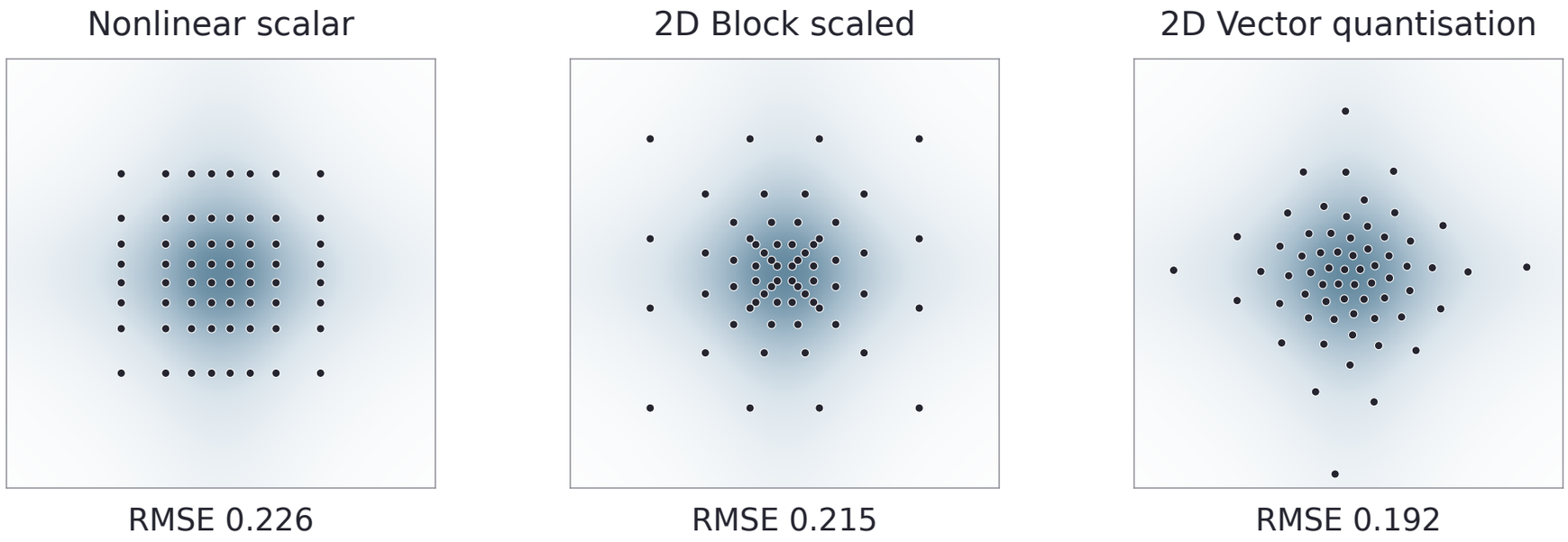


Illustrative tradeoff analysis between size and accuracy.

Format design : Efficient formats

As a side quest during this project, we worked on a paper: Optimal Formats for Weight Quantisation.[1]

One takeaway from this was that **non-linear scalar formats have an efficiency ceiling**, which matters most at extreme compression and for heavy-tailed data.



Quantisation of IID Student's t (df = 7) data to 3 bits/value. **Nonlinear scalar**: Fits 2^3 = 8 centroids (plotting the cartesian product). **2D block**: One 2-bit scale per two 2-bit values. **2D vector quantisation**: Fits 2^6 = 64 centroids. All scales & centroids are fitted to minimise RMSE.

[1] On reflection, the title was a bit of an overshoot.

Format design : Target platform

We want to optimise:

```
output = input @ dequantise(weight).T
```

...on an Android phone with Arm cores (e.g. Pixel 8a).

Since it has the `i8mm` extension, a fast baseline dot product reads `weight` directly as `INT8` and runs a channel-scaled dot product.

Relevant to nonlinear/vector formats, **TBL** performs **16 parallel byte lookups** using **6-bit indices** into a 64-byte table:

```
# TBL operation
table: byte[64]    # (four registers)
index: uint8[16]   # in 0..63
value: byte[16] = table[index]
```

Baseline: Illustrative INT8 matrix-vector product with a per-token input scale and per-output weight scales:

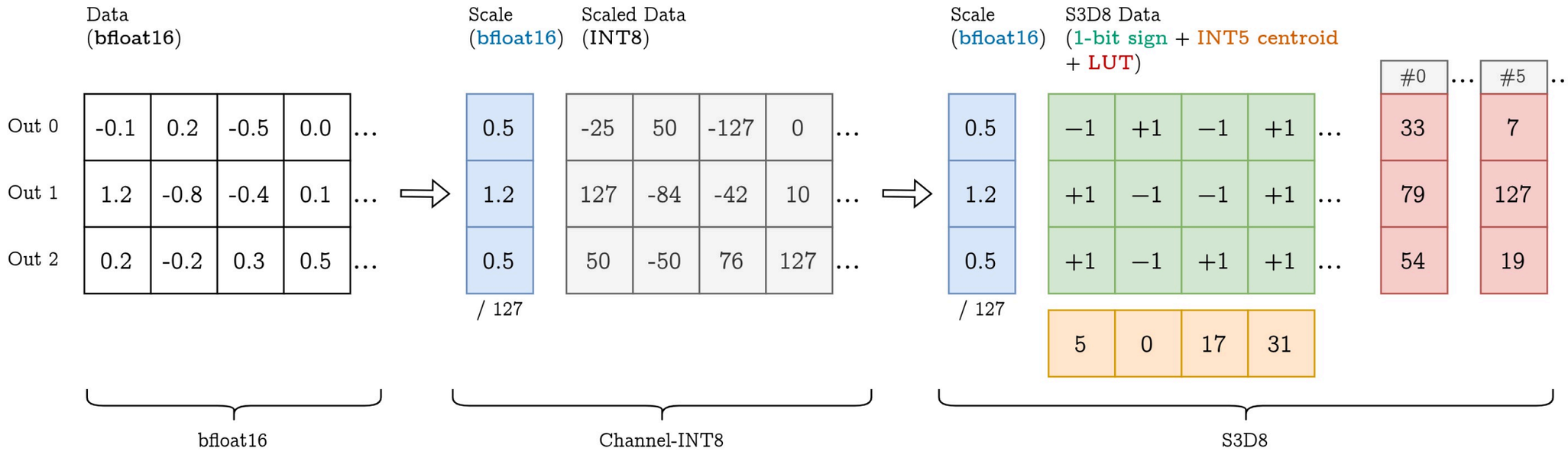
```
for i in range(n_output):
  acc: int32 = 0
  for j in range(n_input):
    acc += input_int8[j] * weight_int8[i, j]

  output[i] = (
    float(acc)
    * input_scale
    * weight_scale[i]
  )
```

Format design : S3D8

S3D8 is a 3D absolute-value vector format with separate sign bits, with an average ~2.7 bits/weight. It decodes to channel-scaled INT8 for compute. Specifically:

- **absolute-value centroids**: a 32 × 3 table of positive INT8 values (per matrix)
- **channel scales**: a bfloat16 scale (per output channel)
- **shared centroid indices**: a 5-bit index (per 3 weights)
- **sign-selection information**: 3 bits (per 3 weights), representing one sign selector per weight



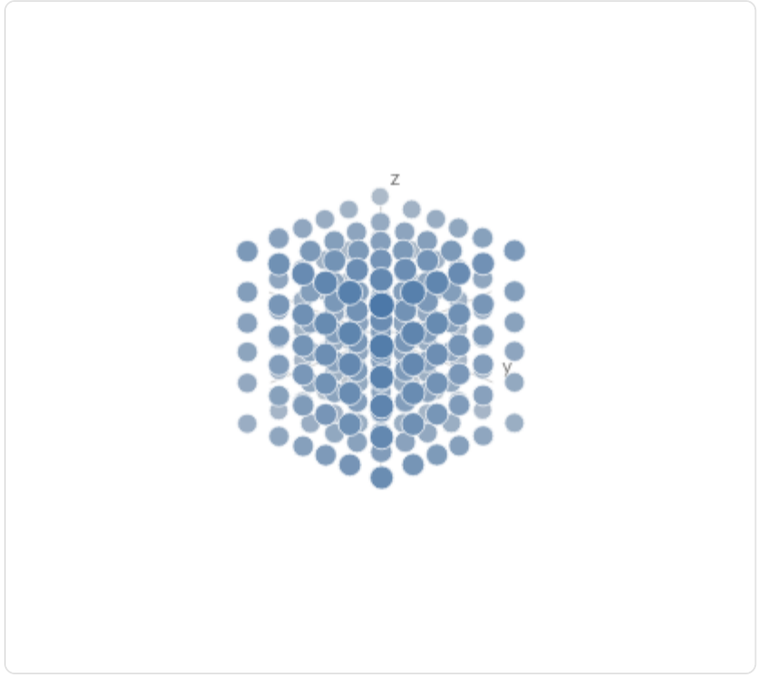
Format design : S3D8 centroids

Examples on artificial IID data.

Scalar · 1D(8/3)

6 values per axis, 216 vectors

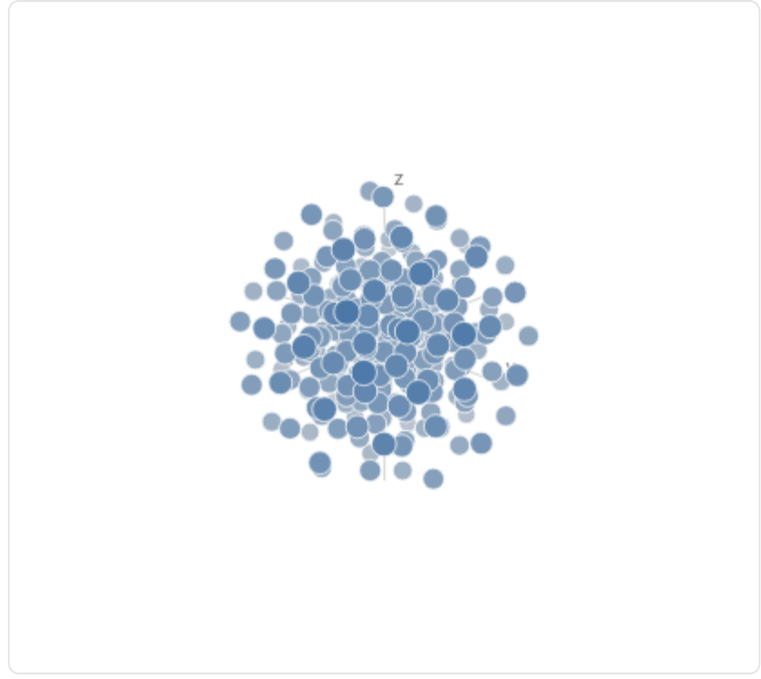
RMS error: 0.2408 **Points:** 216



Vector · 3D8

256 unconstrained centroids

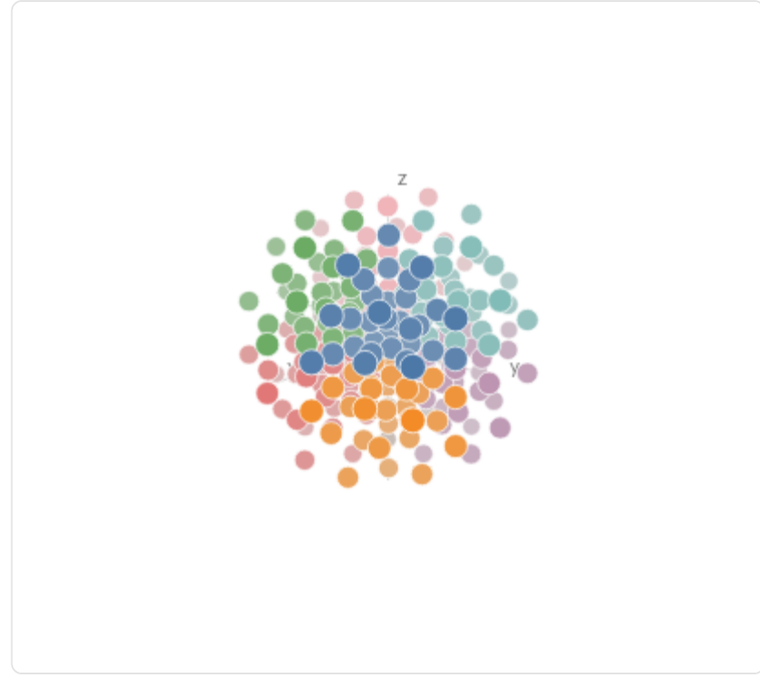
RMS error: 0.2042 **Points:** 256



Symmetric vector · S3D8

32 centroids × 8 sign patterns

RMS error: 0.2094 **Points:** 256

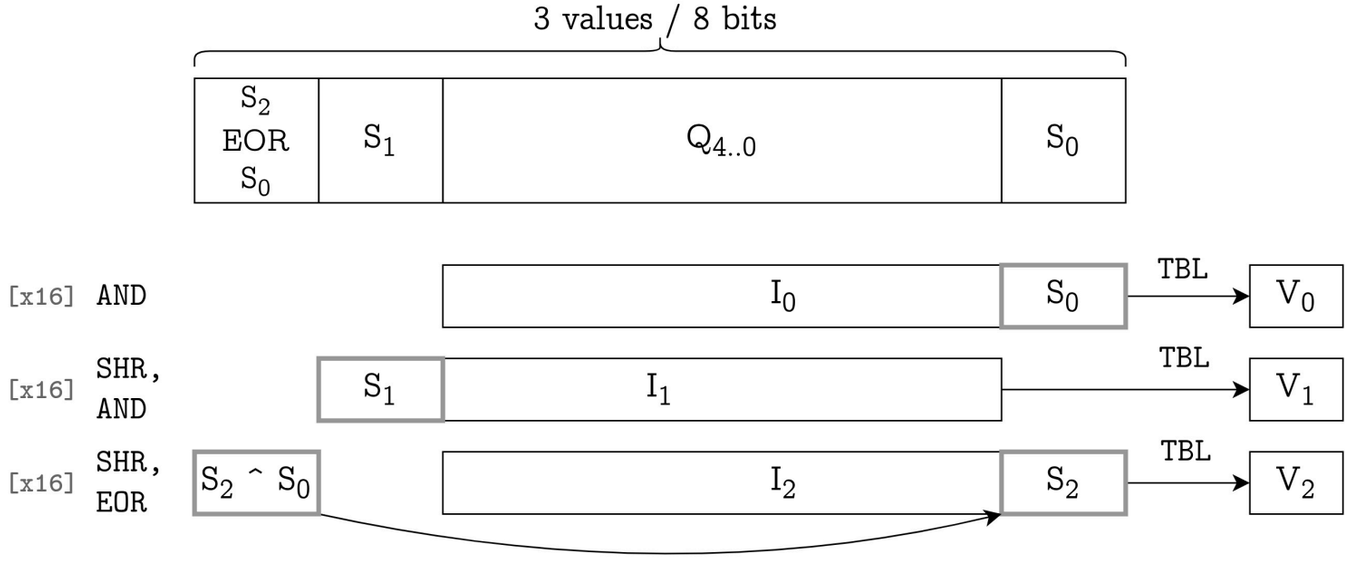


Weight distribution: Normal

Format design : S3D8 decoding

Turn the 32 × 3 × `uint8` absolute-value LUT into three 64 × `int8` signed LUTs. This lets TBL select the value and sign in one lookup.

Layout tweak: Arrange bits to minimise shifting and masking. Set sign-minor/sign-major LUT ordering to match.



Q: shared 5-bit centroid index. **S_i**: sign bit for weight i. **I_i**: 6-bit signed-LUT index. **V_i**: decoded INT8 weight.

Decoding excerpt (48 weights in 8 SIMD instructions):

```
//  v5: 0x3f mask
// v26: 16 packed bytes

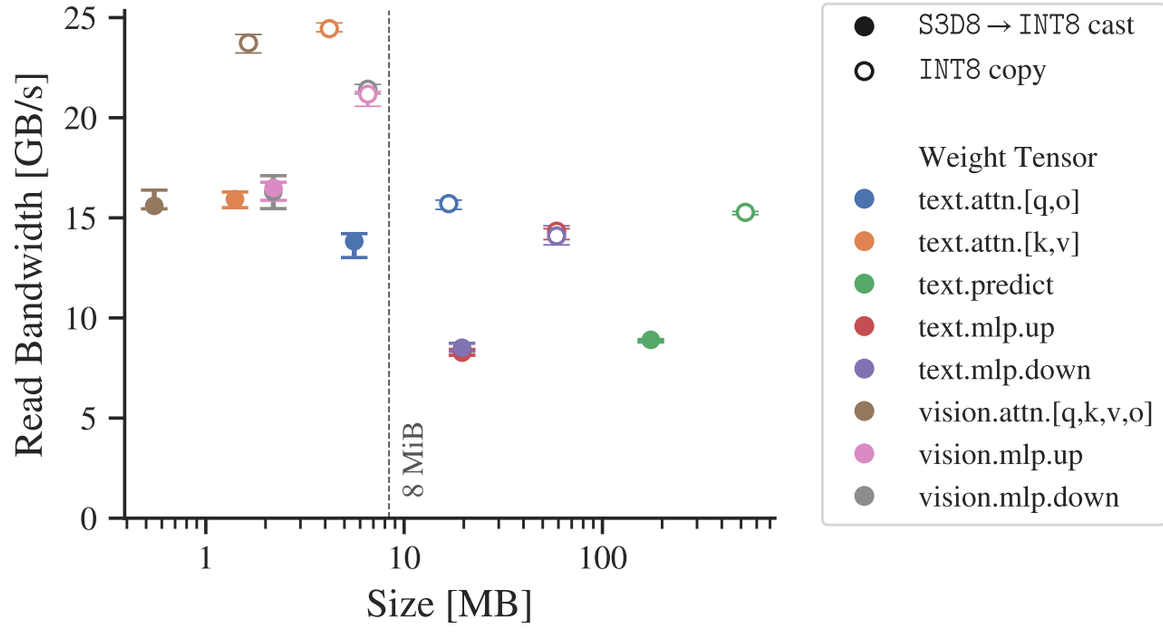
and  v27.16b, v26.16b, v5.16b
ushr v28.16b, v26.16b, #1
and  v28.16b, v28.16b, v5.16b
ushr v26.16b, v26.16b, #7
eor  v26.16b, v27.16b, v26.16b

tbl  v29.16b, {v1.16b-v4.16b}, v27.16b
tbl  v28.16b, {v16.16b-v19.16b}, v28.16b
tbl  v26.16b, {v20.16b-v23.16b}, v26.16b
```

Format design : Decoding speed

Microbenchmarks show lower read bandwidth utilisation for S3D8 decoding versus a plain copy (as expected). However, since S3D8 decoding reads roughly one-third as many bytes as INT8 copying, it is faster per-tensor (writes to cache).

E.g. for the vision MLP up-projection, S3D8 decoding takes **133 µs**, versus **310 µs** for an INT8 copy.



Microbenchmark of S3D8 → INT8 decoding versus INT8 copying on five Pixel 8a cores. Bandwidth counts source bytes only; S3D8 decoding writes three times as many bytes as it reads. Error bars: 95% confidence intervals.

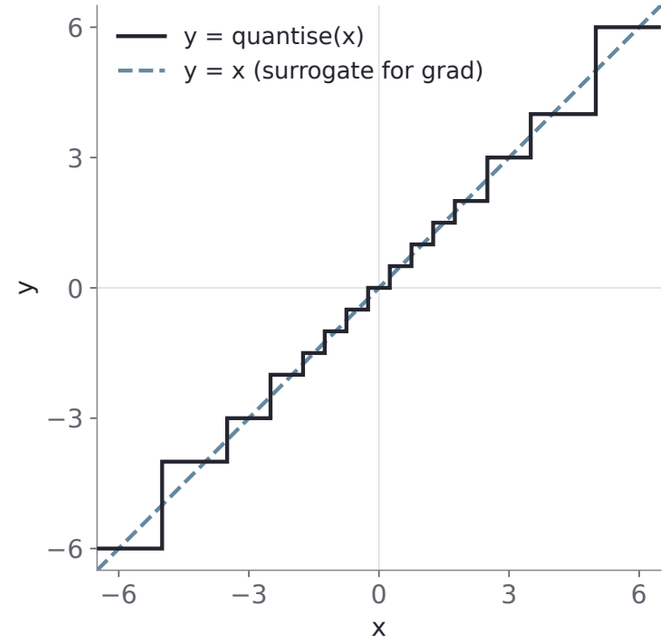
Quantisation-Aware Training (QAT)

Instead of simply casting weights to closest quantised values, we can improve performance using QAT.

```
# Given weights, dataset
# quantise_ste() quantises in fwd, identity in bwd

q_weights = weights.clone()
optimiser = Optimiser(q_weights)

for inputs in dataset:
  ref_logits = model(inputs, weights.detach())  # no grad
  q_logits = model(inputs, quantise_ste(q_weights))
  loss = kl_div(q_logits, ref_logits)
  loss.backward()
  optimiser.step()
```



Straight-through estimator (STE), illustrated for FP4 (E2M1).

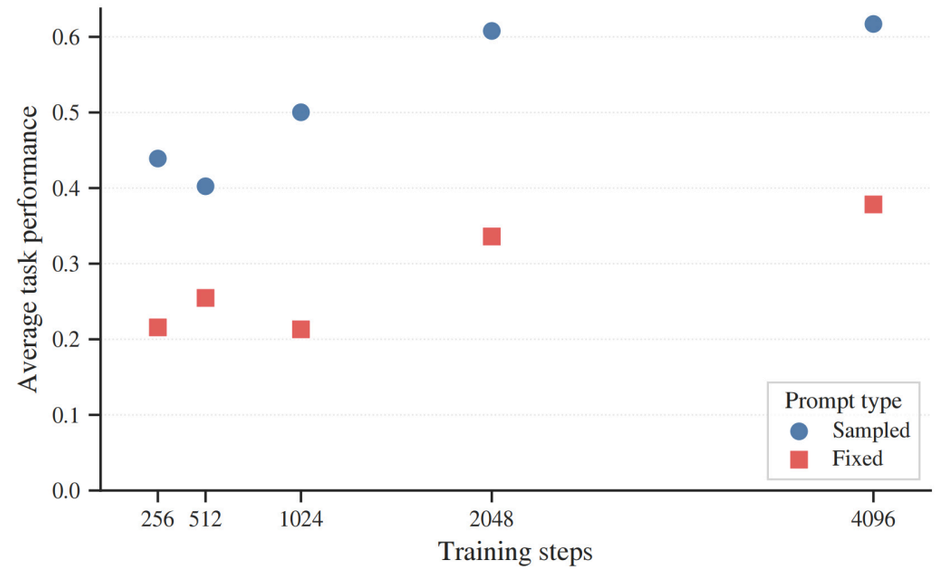
QAT : Inputs matter!

We don't have Meta's training or post-training dataset for Llama.

So, use the (unquantised) model to sample text sequences, following LLM-QAT.

But, choosing the right synthetic data generation procedure matters massively (more than we expected). We needed to sample:

- **images** (from ImageNet),
- diverse **questions**,
- **instructions** which guide the model to produce shorter or longer responses, or change output format,
- **prompt format**: simple or instruction-tuned.

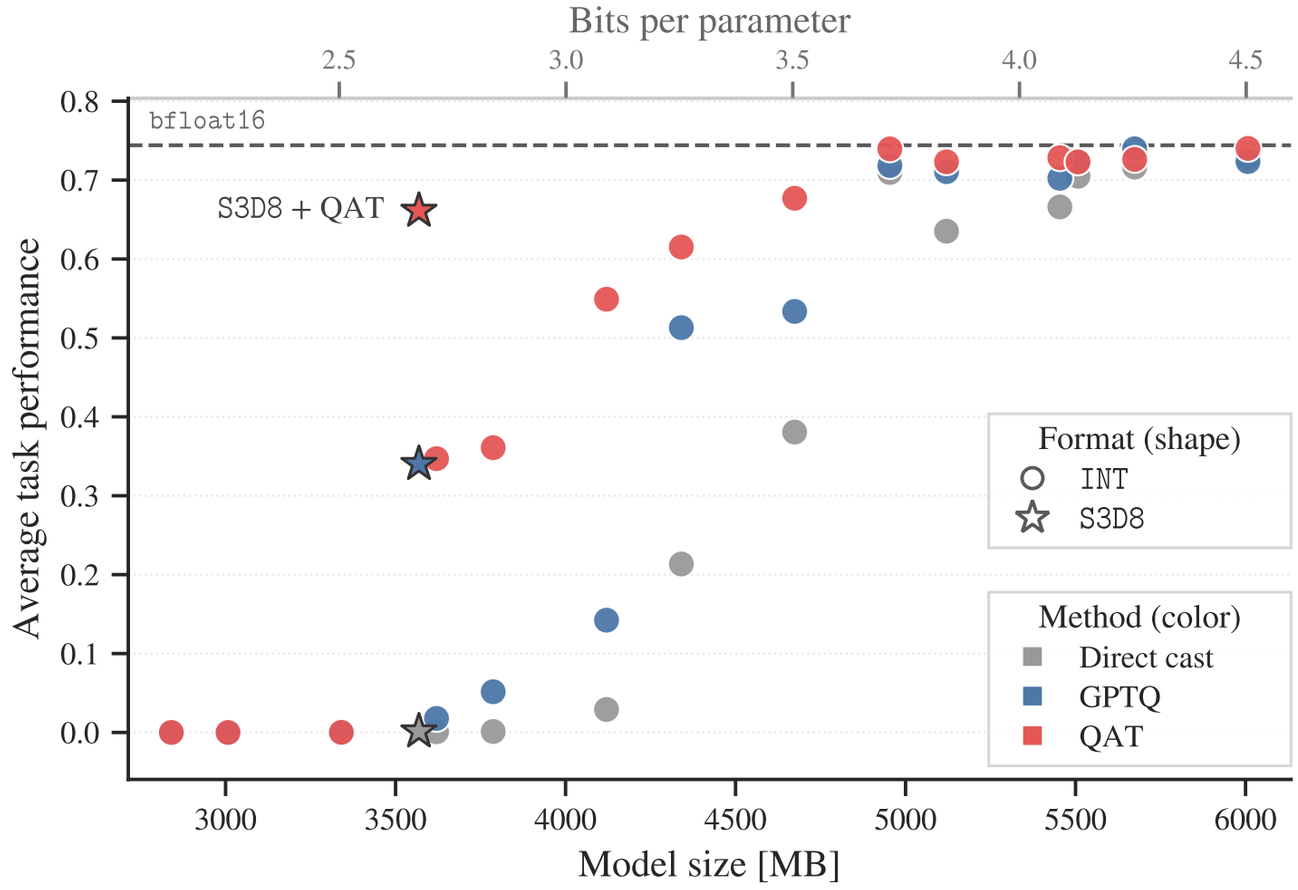


Sampling diverse prompts improves QAT performance compared with the fixed prompt "describe the image:".

Main result

Compare quantised model size against average performance on VQAv2, ChartQA, DocVQA and AI2D.

- **QAT partly recovers performance** lost to aggressive quantisation.
- **S3D8 gives a ~22% smaller model** versus block-scaled INT at matched task performance under QAT.
- **GPTQ is strong above ~3.5 bits/weight**, but trails QAT below this.



Average score on four tasks, using fixed 1,024-example subsets. Dashed line: original bfloat16 model.

Limitations and further work

Limitations

- A non-negligible task performance degradation remains.
- Evaluation covers only one model.
- No comparison with common llama.cpp quantisation formats.
- No GPU kernels for prefill.
- The model fits on a phone, but remains too slow/heavy for practical deployment.

Further work

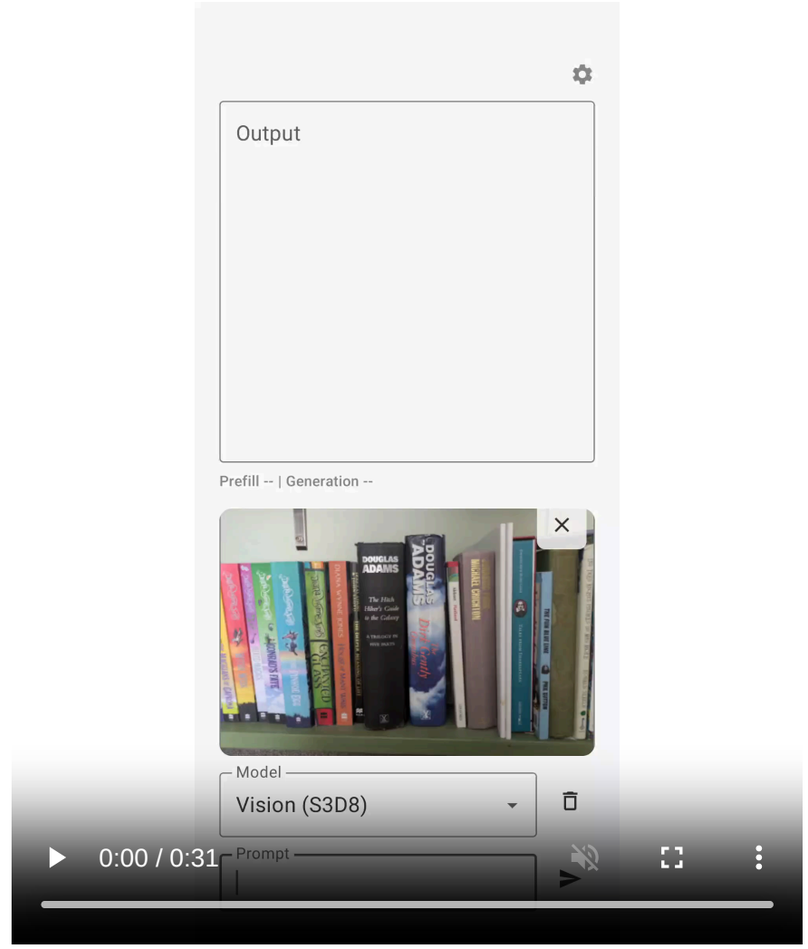
- Broaden model and format comparisons, and add GPU prefill support.
- Explore stronger vector formats, such as trellis-coded quantisation.

Conclusion & Question time

It worked! 😅 We fit an **11B-parameter VLM on a phone** using S3D8 and self-distillation QAT.

- **Vector formats** with kernel–format codesign give compact weights and fast decoding via SIMD register-based lookups.
- **QAT needs care**, as synthetic data can steer the model away from its instruction-tuned behaviour.

Questions welcome!



Llama 3.2 Vision 11B running on a Pixel 8a.

[the paper, blog]